LEBench on one virtual machine

SOSP 2019, Ren et al.

Haoyu Zou 5260000157

CS340 Operating Systems

3 November 2026

What the paper claims

- Core kernel operations got slower from Linux 3.0 to 4.20 (Figure 1).
- Big write and big munmap are the exceptions.
- Eleven causes cover the jumps: security, features, and bad config (Table 3, Figure 2).
- Turning them off speeds Redis, Apache, and Nginx (Section 5).

We test the first claim on the authors' kernel list, and one security switch as the extension. The applications are not run.

What actually runs

- Unchanged measurement code from LEBench commit `6aa7801`.
- Same K-best rule: five samples within 5%, timed with `clock_gettime`.
- Shorter loops (`BASE_ITER` 40) so each boot can be repeated three times.
- Ubuntu 16.04.6 in VirtualBox, legacy BIOS, disk on `D:`.
- One `grub-reboot` per kernel. A panic falls back to 4.4.0-210.

Why the numbers are not the paper's numbers

- Guest: 2 vCPUs, 4 GB, virtual disk, memory integrity on the host.
- CPU: i7-13700H. Meltdown: not affected.
- The paper's server is a Xeon E5-2630 v3 with 128 GB.
- We compare kernels to each other on this guest. We do not match the paper's nanoseconds.

Extension: one switch, `nopti`

Same kernel (4.4.0-210), same binary, command line only.

- getpid 314 ns vs 295 ns. The intervals overlap.
- small read, send, recv, select, poll, and epoll also overlap.
- fork 93 μs vs 254 μs. The intervals do not overlap, and the direction is the wrong one for a KPTI tax.
- `/sys/.../meltdown` says "Not affected" on both boots.

The full table is in the report.

What `nopti` shows

- The vulnerability file says this CPU is not affected by Meltdown.
- getpid, read, send, and recv move by less than the run-to-run spread.
- fork is slower with `nopti`, which is not the paper's KPTI story.
- The KPTI tax is not reproduced here. The CPU never pays it.

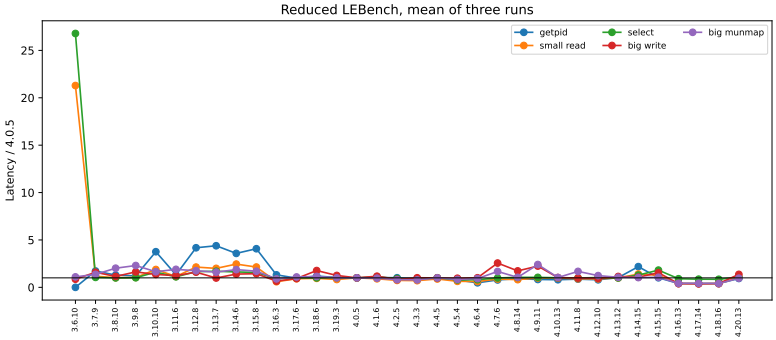
Extension: batch `getpid`

One change. Time 1,000 calls, divide by 1,000. Same K-best, same machine.

- 4.0.5: 245 $\pm$ 0 ns down to 129 $\pm$ 0 ns. The intervals do not overlap.
- 4.4.0-210: 314 $\pm$ 39 ns down to 127 $\pm$ 40 ns.
- After batching, those two kernels overlap. The single-shot slowdown does not.
- The empty clock pair on 4.0.5 is about 130 ns. The old `getpid` row was mostly that pair.

Across kernels

Across 627 later-kernel measurements, 271 (43%) have a higher mean than the same test on 4.0.5 and 76 (12%) have a mean at least 50% higher. 117 of those measurements have non-overlapping mean±stdev intervals and a higher mean, which is the bar used in the analysis.



A rewrite we did not use

A second C suite reimplemented the tests with `rdtsc` and had an empty result table.

- `rdtsc` is the wrong clock on this hypervisor.
- `select` allocates inside the timed region.
- Context switch is one average, not K-best, and is not run at priority -20.
- It had not been executed on Linux.

The course tree calls `OS_Eval` instead of replacing it.

Verdict

34 of 41 listed kernels finished three runs. 3.0.7–3.5.4 never answered sshd. 4.19.11 had no published modules package.

- Claim 1: not supported here. 43% of later-kernel means are higher than our 4.0.5; 12% are at least 50% higher.
- getpid is clearly faster on 9 of 19 later kernels, and slower on 2.
- Claim 2: `nopti` does not reproduce the KPTI tax. This CPU is not affected.
- Claim 3: not run, so inconclusive.
- Overall: partially reproduced. The missing piece is the paper's own conclusion, not the harness.

What failed quietly

- A copied benchmark still prints a number when the timer is wrong.
- A kernel package can install and then never reach sshd.
- Both have to be in the log: a CSV, or a boot-failure line.
- Upstream `grub-install` onto a partition is not used. It can overwrite the wrong disk.